SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 20%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Industry Problem Solving

Code of Conduct:

I Cheredla Jasvina/192324026certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1, A retail company wants to recommend products to customers based on past purchases. Design a solution using K-Nearest Neighbour or Case-Based Learning. Explain how similarity is measured, how recommendations are generated, and how the system adapts to new customer data.

Answer :

Product Recommendation System using K-Nearest Neighbour (KNN)

A retail company can build a product recommendation system using K-Nearest Neighbour (KNN). The system recommends products by finding customers whose purchasing behaviour is most similar to the current customer.

1. Dataset

Suppose the company stores customer purchase information:

Customer	Electronics	Clothing	Books	Groceries	Purchased Product
C1	5	2	1	3	Headphones
C2	4	1	2	4	Smart Watch
C3	1	5	3	2	Jacket
C4	5	2	2	3	Earbuds
C5	1	4	4	2	Shoes

Each customer's purchase history is represented as a feature vector.

For example:

C1 = [5, 2, 1, 3]

New customer:

Cnew = [5, 2, 2, 3]

2. Similarity Measurement

KNN can use Euclidean distance to measure similarity.

For the new customer and C1:

A smaller distance means greater similarity.

3. Generating Recommendations

Suppose:

K = 3

The three most similar customers are:

- C1 → Headphones
- C2 → Smart Watch
- C4 → Earbuds

The system examines the products purchased by these nearest customers.

If the most frequently occurring products are Headphones and Earbuds, they are recommended to the new customer.

The recommendation process is:

New Customer

↓

Purchase History

↓

Calculate Distance

↓

Find K Nearest Customers

↓

Analyze Their Purchases
↓
Rank Products
↓
Recommend Top Products

4. KNN Recommendation Algorithm

- Collect customer purchase histories.
- Convert each customer into a feature vector.
- Receive the new customer's purchase history.
- Calculate the distance between the new customer and all existing customers.
- Sort customers according to distance.
- Select the K nearest customers.
- Collect products purchased by these customers.
- Count the frequency of each product.
- Rank products according to frequency.
- Recommend the top-ranked products.

5. Adapting to New Customer Data

The system can adapt easily because KNN is a lazy learning algorithm.

When a customer makes a new purchase:

New Purchase
↓
Update Customer History
↓
Add/Update Customer Feature Vector
↓
Use Updated Dataset
↓
Generate Future Recommendations

For example, if a customer initially purchases books and clothing but later purchases a laptop, the customer's feature vector is updated. The next time recommendations are generated, KNN considers this new information.

Therefore, the system does not need to completely retrain a complex model every time a new purchase is made.

6. Advantages

- Simple to implement.
- Works well with customer purchase histories.
- Recommendations are based on actual similar customers.
- New customer data can be incorporated easily.
- No complicated model training is required.
- Can provide personalized recommendations.

7. Limitations

- Searching through a very large customer database can be computationally expensive.
- Choosing a suitable value of K is important.
- Features should be normalized when they have different scales.
- A new customer with no purchase history creates a cold-start problem.

Conclusion

A KNN-based recommendation system identifies customers with similar purchasing behaviour using a distance measure such as Euclidean distance. Products purchased by the nearest customers are ranked according to their frequency and recommended to the new

customer. As customers make new purchases, their purchase profiles are updated, allowing the system to continuously adapt and provide more personalized recommendations.

2. A manufacturing industry needs to predict machine output based on sensor readings that vary over time. Propose a solution using Locally Weighted Regression. Explain how local models improve prediction accuracy and how the system handles noisy or incomplete data.

Answer :

Locally Weighted Regression for Machine Output Prediction

A manufacturing industry can use Locally Weighted Regression (LWR) to predict machine output from sensor readings such as temperature, pressure, vibration, speed, and humidity. Since sensor values and machine behaviour can change over time, LWR is useful because it gives more importance to data points that are close to the current operating condition.

1. Input Dataset

The historical dataset can contain:

Temperature	Pressure	Vibration	Speed	Machine Output
60	20	2.1	1000	80
65	22	2.3	1050	84
70	24	2.5	1100	88
75	26	2.8	1150	91
80	30	3.2	1200	86

The sensor readings are the input features, and machine output is the target variable**.**

2. Working of LWR

When a new sensor reading arrives, LWR calculates the distance between the new reading and historical observations.

A Gaussian kernel can be used to calculate the weight:

where:

- = weight of the data point
- = distance from the query point
- = bandwidth
- Smaller distance → larger weight
- Larger distance → smaller weight

Thus, sensor readings that represent operating conditions similar to the current machine condition have greater influence on the prediction.

3. Local Model

Instead of fitting one regression model to the entire dataset, LWR creates a local regression model around the query point.

For example, if the current machine condition is:

Temperature = 72
Pressure = 25
Vibration = 2.6
Speed = 1120

LWR gives higher weights to historical observations close to these values.

The nearby observations strongly influence the predicted machine output, while distant observations have little influence**.**

4. How Local Models Improve Accuracy

A single global regression model assumes that the relationship between sensors and machine output is approximately the same everywhere.

However, manufacturing machines may behave differently under different operating conditions.

For example:

Low Temperature → Normal Output
Medium Temperature → Increasing Output
High Temperature → Output Starts Decreasing

A global model may not capture all these changes accurately.

LWR instead fits a model around the current operating region. Therefore:

Global Regression:

All historical data
↓
One regression model
↓
Prediction

Locally Weighted Regression:

Current sensor readings
↓
Find nearby observations
↓
Assign higher weights to nearby data
↓
Build local regression model
↓
Predict machine output

This makes LWR suitable for systems where machine behaviour changes over time.

5. **Handling Noisy Data**

Manufacturing sensors may contain noise because of:

- Sensor errors
- Mechanical vibration
- Electrical interference
- Sudden temperature changes
- Measurement errors

LWR reduces the effect of distant noisy observations because their weights become small.

For example:

Nearby reliable data → High weight → Strong influence
Distant noisy data → Low weight → Small influence

The bandwidth can also be adjusted.

- Small bandwidth: Uses a very small local region and can follow rapid changes, but may be sensitive to noise.
- Large bandwidth: Uses more observations and produces smoother predictions, but may miss local changes.
- Suitable bandwidth: Provides a balance between noise reduction and local accuracy.

6. Handling Incomplete Data

If some sensor values are missing, the system can use preprocessing before applying LWR.

Possible methods include:

- Mean/median imputation
- Interpolation using previous and next readings
- Forward filling for time-series sensor data
- Using only available sensor features when appropriate

For example:

Temperature = 72
Pressure = Missing
Vibration = 2.6
Speed = 1120
The missing pressure value can be estimated from nearby time readings or historical sensor values before calculating the LWR prediction.

7. Advantages

- Adapts to changing machine behaviour.
- Gives greater importance to similar operating conditions.
- Can model nonlinear relationships better than a single global linear model.
- Reduces the influence of distant observations.
- Useful for time-varying sensor data.
- Can provide accurate local predictions.

8. Limitations

- Prediction can be computationally expensive for a large dataset because distances and weights must be calculated for each query.
- Choosing the correct bandwidth is important.
- Very small bandwidth may cause overfitting.
- Very large bandwidth may make the model behave like global regression.
- Missing sensor values need preprocessing.

Conclusion

Locally Weighted Regression is suitable for predicting manufacturing machine output when sensor readings vary over time. It calculates the similarity between the current machine condition and historical observations, assigns higher weights to nearby observations, and builds a local regression model for prediction. Local models can improve accuracy when machine behaviour changes across different operating conditions, while appropriate bandwidth selection and preprocessing help handle noise and incomplete sensor data.

3. A healthcare company aims to classify patients into risk categories using historical medical data. Suggest a model using KNN or RBF networks. Explain how the algorithm handles large datasets, feature selection, and accuracy improvement in real-world scenarios.

Answer :

Patient Risk Classification using KNN

A healthcare company can use a K-Nearest Neighbour (KNN) model to classify patients into risk categories such as Low Risk, Medium Risk, and High Risk based on historical medical data.

1. Dataset

The dataset can contain features such as:

Patient	Age	Blood Pressure	Glucose	BMI	Risk
P1	25	120	90	22	Low
P2	35	130	110	25	Low
P3	50	145	150	30	Medium
P4	60	160	180	34	High
P5	70	170	200	36	High

For a new patient, KNN compares the patient's medical features with historical patients.

2. Working of KNN

The steps are:

Historical Patient Data

↓

Data Preprocessing
↓
Feature Selection
↓
Calculate Distance
↓
Find K Nearest Patients
↓
Majority Voting
↓
Risk Category

For example, if:

K = 3

Nearest patients:

Patient 1 → Medium Risk

Patient 2 → High Risk

Patient 3 → High Risk

The majority class is **High Risk**, so the new patient is classified as **High Risk**.

3. Similarity Measurement

Euclidean distance can be used:

A smaller distance means that two patients have more similar feature values.

Before calculating distance, features should normally be **standardized**, because values such as glucose, age, and blood pressure have different numerical scales.

4. Handling Large Datasets

KNN can become slow when the healthcare database contains millions of patients because it compares a new patient with many stored patients.

To improve performance:

- Use efficient data structures such as **KD-Trees** or **Ball Trees**.
- Remove unnecessary features.
- Use dimensionality reduction when appropriate.
- Store and search only relevant patient records.
- Use approximate nearest-neighbour methods for very large datasets.

Thus, the system can find similar patients without comparing against every record unnecessarily.

5. Feature Selection

Not every medical feature contributes equally to risk prediction.

Important features can be selected using:

- Correlation analysis
- Mutual information
- Feature importance
- Recursive Feature Elimination (RFE)
- Domain knowledge from medical experts

For example, if glucose, blood pressure, BMI, and age are strongly related to the target risk category, these features can be given priority.

Removing irrelevant features improves both **prediction accuracy and computation speed**.

6. Improving Accuracy

The accuracy of KNN can be improved by:

1. Feature Scaling

Normalize or standardize numerical features so that one feature does not dominate the distance calculation.

2. Selecting the Correct K
A very small K may make the model sensitive to noise.
A very large K may make the model too general.
The best K can be selected using **cross-validation**.
3. Removing Noisy Data
Incorrect or abnormal records can negatively affect nearest-neighbour classification.
4. Handling Missing Values
Missing medical values can be handled using suitable imputation techniques such as mean, median, or model-based imputation.
5. Class Imbalance
If most patients belong to the Low Risk category, the model may be biased toward that class. Techniques such as class balancing or suitable sampling methods can help.
7. Real-World Example
Suppose a new patient has:
Age = 58
Blood Pressure = 155
Glucose = 175
BMI = 32
The system compares these values with historical patients and finds:
Patient A → High Risk
Patient B → High Risk
Patient C → Medium Risk
Patient D → High Risk
Patient E → Medium Risk
If **K = 5**:
High Risk = 3
Medium Risk = 2
Low Risk = 0
Therefore:
Predicted Risk = High Risk
8. Advantages
- Simple and easy to understand.
- Works well for classification problems.
- No complex training process is required.
- Can classify new patients using similar historical cases.
- Can be updated easily when new patient data becomes available.

9. Limitations
- Prediction can be slow for very large datasets.
- Sensitive to irrelevant features.
- Sensitive to feature scaling.
- Choice of K affects accuracy.
- Missing and noisy medical data can affect predictions.

Conclusion
A **KNN-based patient risk classification system** can classify patients into Low, Medium, or High Risk by comparing their medical features with similar historical patients. Feature selection and normalization improve similarity calculations, while efficient nearest-neighbour search helps handle large datasets. Cross-validation, appropriate K selection, noise removal, missing-value handling, and class balancing can further improve accuracy in real-world healthcare applications.

4. An e-commerce platform wants to detect similar products for dynamic pricing strategies. Design a solution using Case-Based Learning. Discuss how similarity metrics are defined, how past cases are reused, and how the system scales with increasing product data.

Answer :

Case-Based Learning for Similar Product Detection

An e-commerce platform can use **Case-Based Learning (CBL)** to identify products that are similar to a new product and use the pricing information of past similar products for dynamic pricing.

1. Product Dataset

The system stores previous product cases containing product attributes and pricing information.

Product	Category	Price	Rating	Sales	Brand
P1	Laptop	60000	4.5	500	A
P2	Laptop	65000	4.3	450	B
P3	Mobile	30000	4.6	800	A
P4	Laptop	62000	4.4	550	C

Each product is represented using features such as category, price, rating, sales, brand, specifications, and customer demand.

2. Similarity Metrics

The system compares a new product with stored products using a similarity measure.

For numerical features, **Euclidean distance** can be used:

For categorical features such as product category or brand, **matching similarity** can be used.

The overall similarity can be calculated as:

where represents the importance of each feature.

A higher similarity score means that the products are more similar.

3. Reusing Past Cases

When a new product enters the system:

New Product
↓
Extract Product Features
↓
Compare with Previous Products
↓
Find Most Similar Cases
↓
Retrieve Their Pricing Information
↓
Analyze Demand and Sales
↓
Suggest Suitable Price

For example, suppose a new laptop is similar to three previous laptops:

Laptop A → ₹60,000 → High Sales

Laptop B → ₹62,000 → Medium Sales

Laptop C → ₹65,000 → High Sales

The system analyzes these previous cases and may suggest a price such as **₹62,000–₹64,000**, depending on current demand and business rules.

4. Case Adaptation

CBL does not simply copy the old price. It adapts the previous case to the current situation.

For example:

Past Case:
Laptop → ₹60,000 → High Sales
Current Situation:
Similar Laptop
High Demand
Low Stock
↓
Adjusted Price
↓
₹63,000
Thus, historical knowledge is reused while considering current market conditions.

5. Handling Increasing Product Data

As the number of products increases, comparing a new product with every stored case becomes expensive.

The system can scale by using:

- **Database indexing** to quickly retrieve relevant products.
- **Feature-based filtering** to search only within the same category.
- **Clustering** to group similar products.
- **Vector databases** for fast similarity searches.
- **Approximate nearest-neighbour search** for large product collections.
- Removing outdated or irrelevant cases.

For example:
Millions of Products
↓
Category Filtering
↓
Feature/Vector Search
↓
Top Similar Products
↓
Pricing Analysis

6. Updating the Case Base

Whenever a product is sold or its price changes, the new information can be stored as a new case.
New Product Data
↓
New Case
↓
Case Base
↓
Future Recommendations
This allows the system to continuously learn from new pricing and sales behaviour.

7. Advantages

- Reuses previous pricing experiences.
- Easy to update with new product cases.
- Provides product-specific pricing recommendations.
- Can adapt to changing demand.
- Does not require complete model retraining whenever new cases are added.

8. Limitations

- Searching a very large case base can be computationally expensive.

- Poor-quality historical cases can produce poor recommendations.
- Similarity weights must be chosen carefully.
- Product prices and market conditions change frequently.
- New products with no similar historical cases create a **cold-start problem**.

Conclusion

A **Case-Based Learning system** can detect similar products by comparing their features using appropriate similarity metrics. The system retrieves relevant historical cases, analyzes their prices, sales, and demand, and adapts the previous pricing solution to the current product. As the product database grows, indexing, clustering, vector search, and approximate similarity techniques can be used to maintain fast and scalable performance.

5. A logistics company needs to forecast delivery times based on location and traffic patterns. Propose a solution using Radial Basis Function networks or Locally Weighted Regression. Explain how the model captures non-linear relationships and improves prediction reliability under varying conditions.

Answer :

Delivery Time Forecasting using Radial Basis Function (RBF) Network

A logistics company can use an **RBF Network** to predict delivery time based on location, distance, traffic, weather, vehicle speed, and other relevant factors. RBF networks are suitable because delivery time often has a **non-linear relationship** with these factors.

1. Input Dataset

Historical delivery data can contain:

Distance (km)	Traffic Level	Average Speed (km/h)	Weather	Delivery Time (min)
5	1	45	1	15
10	2	35	1	25
15	3	25	2	45
20	4	20	2	65
25	5	15	3	90

The inputs are used to predict the **delivery time**.

2. RBF Network Structure

Location + Distance + Traffic + Weather + Speed

↓

Input Layer

↓

Gaussian RBF Layer

↓

Weighted Sum

↓

Predicted Delivery Time

The RBF layer contains several Gaussian functions called **RBF units**.

3. Gaussian Function

The Gaussian function is:

where:

- = current input
- = RBF center
- = width of the Gaussian function
- = activation value

If the current traffic and location conditions are close to an RBF center, the activation is high. If they are far away, the activation is low.

4. Capturing Non-Linear Relationships

Delivery time does not always increase linearly with distance.

For example:

Low Traffic + Short Distance

↓

Fast Delivery

High Traffic + Short Distance

↓

Delayed Delivery

High Traffic + Long Distance

↓

Very Long Delivery

The combination of distance and traffic can create complex patterns.

RBF networks handle this by using multiple Gaussian functions. Each function learns a different region of the input space.

For example:

RBF Center 1 → Low Traffic Area

RBF Center 2 → Medium Traffic Area

RBF Center 3 → Heavy Traffic Area

RBF Center 4 → Long Distance Area

The outputs from these functions are combined to produce the final delivery-time prediction.

5. Improving Prediction Reliability

The model can be trained using historical delivery records from different:

- Locations
- Traffic conditions
- Times of day
- Weather conditions
- Distances
- Vehicle types

This allows the network to learn different delivery patterns.

For example, during normal traffic:

10 km → 25 minutes

During heavy traffic:

10 km → 40 minutes

The RBF network can learn these different relationships instead of assuming one fixed relationship.

6. Handling Varying Conditions

The system can receive real-time information such as:

Current Location

+

Distance

+

Traffic

+

Weather

+

Time of Day

↓

RBF Network

↓
Predicted Delivery Time
If traffic suddenly increases, the new sensor/input values activate different RBF units, allowing the prediction to change accordingly.

7. Improving Accuracy

Prediction reliability can be improved by:

- Using more historical delivery data.
- Selecting suitable RBF centers.
- Selecting an appropriate Gaussian width.
- Normalizing input features.
- Removing incorrect or noisy records.
- Including important traffic and location features.
- Continuously updating the model with new delivery records.
- Evaluating the model using MAE, MSE, or RMSE.

8. Advantages

- Captures complex non-linear relationships.
- Works well with changing traffic conditions.
- Can combine multiple input variables.
- Produces smooth predictions.
- Can adapt to different locations and traffic patterns.
- Useful for real-time delivery-time estimation.

9. Limitations

- Choosing the correct number of RBF centers is important.
- Large datasets can increase training and computation time.
- Poor-quality historical data can reduce accuracy.
- Incorrect Gaussian width selection can cause underfitting or overfitting.
- New locations with very little historical data can be difficult to predict accurately.

Conclusion

An **RBF Network** is a suitable solution for forecasting delivery times because it can capture the **non-linear relationship between location, distance, traffic, weather, and delivery time**. Gaussian functions identify similar operating conditions, and their weighted outputs generate the final prediction. By training the network with diverse historical data and continuously incorporating new delivery information, the logistics company can produce more reliable predictions under changing traffic and environmental conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**25**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand